

Atlantic Meat Pre-Production Test Checklist on Localhost

This checklist is to manually verify the application on localhost before a production migration. It is based on the routes, forms, role guards, responsive components, and workflows currently present in the source code.

Important: Run these tests against a dedicated non-production Cognito user pool, API, DynamoDB tables, S3 buckets, and notification services. Several tests create, update, or delete real backend records even though the frontend runs on localhost.

How to use this checklist

- ☐ Record the test date, application commit, backend version, browser, device/viewport, tester, and environment below.
- ☐ Use unique test identifiers such as `LOCAL-QA-YYYYMMDD-01` so created records are easy to find and remove.
- ☐ Test with at least one account for each role: `admin`, `manager`, `user`, `maintenance`, and `contractor`.
- ☐ For every mutation, verify all four outcomes: success feedback, correct redirect, list/detail data refresh, and persisted data after a hard reload.
- ☐ For every destructive test, use disposable test data and confirm both the UI record and its backend-associated files/identity are handled correctly.
- ☐ Record failures in the execution log at the end of this document with screenshots, console output, and network request details.

Test run details

Item	Value
Date	
Tester	Fabian Petersen
Git commit / build	
Frontend URL	<code>http://localhost:5173</code>
API environment	
Cognito user pool	
Desktop browser/version	
Mobile browser/device	

Result convention

- `[x]` = passed
- `[ ]` = not run
- Add `FAIL: defect-id` beside a failed test

- Add **N/A: reason** only when the feature is intentionally excluded from the release

1. Environment and smoke test

- ☐ Install dependencies with **npm ci** without errors.
- ☐ Start the application with **npm run dev** and open the localhost URL printed by Vite.
- ☐ Confirm the app has valid **VITE_COGNITO_USERPOOL_ID**, **VITE_COGNITO_CLIENT_ID**, and **VITE_SITE_URL** values without exposing them in the browser UI or repository.
- ☐ Open the app in a clean/incognito session; the login screen renders without a blank page or uncaught console error.
- ☐ Confirm the Atlantic Meats logo, fonts, icons, colors, and form controls load without missing assets.
- ☐ Confirm authenticated API calls use the intended localhost test API, never production by accident.
- ☐ Confirm API calls include a Cognito bearer token while S3 presigned uploads do not incorrectly use that token.
- ☐ Refresh each major page and confirm React Router resolves it instead of returning a server 404.
- ☐ Confirm unknown URLs show controlled behavior rather than a blank screen.
- ☐ Run **npm run lint** and record the result.
- ☐ Run **npm run build** and record the result.
- ☐ Inspect the browser console during the full test run for errors, warnings, failed source maps, and leaked sensitive data.
- ☐ Inspect the Network tab for unexpected 4xx/5xx responses, duplicate calls, indefinitely pending calls, and mixed-content/CORS errors.

2. Authentication and session management

Login

- ☐ Submit an empty login form; required/format validation appears and no request is sent.
- ☐ Enter an invalid email; the form displays a useful validation message.
- ☐ Enter a password that does not meet the displayed policy; the form blocks submission appropriately.
- ☐ Toggle password visibility; only the display changes and the password value remains intact.
- ☐ Sign in with valid credentials; the user reaches **/dashboard** and authenticated data loads.
- ☐ Sign in with an incorrect password; a safe, understandable error appears without revealing account existence or backend details.
- ☐ Double-click **Sign In** or press Enter repeatedly; only one effective login occurs.
- ☐ While sign-in is pending, the loading state appears and the submit button cannot be used repeatedly.
- ☐ Visit **/** while already authenticated; the public-only guard redirects away from login.
- ☐ Directly visit a protected URL while signed out; access is denied and the user is returned to the login flow.

First login / temporary password

- ☐ Sign in with a newly created user's temporary password; the new-password challenge appears.
- ☐ Test passwords missing uppercase, lowercase, number, or minimum length; each is rejected.
- ☐ Test mismatched new and confirmation passwords; the form rejects them.
- ☐ Complete the password challenge; the user signs in and reaches the dashboard.
- ☐ Sign out and sign in again with the permanent password; the challenge does not repeat.

- ☐ Confirm the user's application status changes from **FORCE_CHANGE_PASSWORD** to **CONFIRMED** after successful Cognito confirmation.
- ☐ Confirm refreshing or retrying the confirmation does not create duplicate user records or corrupt status.

Forgot password

- ☐ Open **Forgot Password** from login and confirm **/forgot-password** loads.
- ☐ Submit an invalid email; inline validation appears.
- ☐ Submit a registered test email; a reset code is delivered and the UI advances to the expected next step.
- ☐ Enter a non-numeric, short, incorrect, expired, and valid six-digit code; each result is handled correctly.
- ☐ Use **Resend Code**; a new code is delivered, rate limits are handled, and no duplicate uncontrolled requests occur.
- ☐ Set and confirm a compliant new password, then log in with it.
- ☐ Verify the old password no longer works.
- ☐ Use **Back/Return** at each stage; navigation works without leaving sensitive form values visible.

Session lifecycle

- ☐ Refresh an authenticated page; the session rehydrates without briefly exposing unauthorized content or making premature API calls.
- ☐ Open two tabs; sign out in one and verify the other no longer permits protected operations after refresh/request.
- ☐ Let the access token expire or simulate a 401; the user is signed out and redirected to the actual login route **/** rather than a missing **/login** page.
- ☐ Click logout; the local/auth session is cleared and browser Back cannot reopen protected content.
- ☐ Confirm tokens, passwords, reset codes, and temporary passwords never appear in URLs, console logs, toast messages, or local UI history.

3. Role-based navigation and authorization

Test both visible navigation and direct URL entry. Hiding a menu item is not sufficient authorization.

Admin

- ☐ Admin sees Dashboard, all maintenance pages, Assets, Asset Verification, Transfers, Disposals, Users, and My Profile as intended.
- ☐ Admin can access admin-only routes: pending job approval, asset register/create/update/history/manual verification, transfer requests, disposal requests, users, and create user.

Manager

- ☐ Manager sees only intended links and does not see admin-only asset/user management links.
- ☐ Manager can access Dashboard, create/open maintenance jobs, transfer/disposal creation and relevant lists, My Profile, and full-screen asset verification where intended.
- ☐ Manager receives an access denial/redirect when directly entering admin-only URLs.

User

- ☐ User sees only intended links and can access Dashboard, create/open jobs, transfers/disposals and My Profile.
- ☐ User receives an access denial/redirect for admin-only routes and asset verification.

Maintenance

- ☐ Maintenance sees its intended maintenance, transfer/disposal, and profile navigation.
- ☐ Maintenance can open and action assigned/in-progress work and access completed jobs.
- ☐ Maintenance receives an access denial/redirect for admin-only routes.

Contractor

- ☐ Contractor sees only intended links, including completed/my jobs and My Profile if contractor self-service is supported.
- ☐ Contractor can access only the job/profile routes explicitly permitted by route guards.
- ☐ Confirm contractor group recognition, sidebar visibility, and direct-route authorization agree; log any mismatch.

Cross-role checks

- ☐ Enter every restricted route directly for every unauthorized role; no protected data flashes before redirect.
- ☐ Modify an API request outside the UI to target another user's/store's record; backend authorization rejects it.
- ☐ A 403 response shows a controlled message and does not sign the user out unnecessarily.
- ☐ Role changes take effect after token/session refresh and do not leave stale elevated UI access.

4. Global layout, navigation, and feedback

- ☐ Desktop sidebar sections expand/collapse and every visible link opens the correct page.
- ☐ Active navigation styling follows the current route.
- ☐ Mobile menu opens, closes by its control and overlay, scrolls when long, and closes after navigation.
- ☐ Navbar shows the correct signed-in user's name/avatar details.
- ☐ Theme toggle changes light/dark theme, persists as intended, and all text/controls remain readable.
- ☐ Breadcrumbs, Back buttons, Cancel buttons, and page headings lead to the expected parent page.
- ☐ Search button/input appears only on intended pages and searches the currently displayed data correctly.
- ☐ Empty lists show a useful empty state rather than a broken table/card.
- ☐ Loading skeletons/spinners appear during slow requests and disappear on completion.
- ☐ API failures show a useful error state with no infinite spinner.
- ☐ Success/error dialogs or toasts are accurate, dismissible, and do not persist into unrelated actions.
- ☐ Repeated clicks on submit/delete/approve do not create duplicate actions.
- ☐ Keyboard focus is visible and returns sensibly after dialogs/sidebars close.

5. Dashboard

- ☐ `/dashboard` loads all metric cards without console or schema errors.

- ☐ Maintenance jobs, maintenance cost, open requests, and verification charts render with real test data.
- ☐ Dashboard totals agree with the corresponding filtered lists and known test records.
- ☐ Monthly/year-to-date values use the correct year, month, currency, and timezone.
- ☐ Percentage changes handle zero previous values without NaN, Infinity, or misleading output.
- ☐ Empty metrics render as zero/empty states instead of crashing.
- ☐ Dashboard refreshes after a related job/asset workflow mutation or after a hard reload.
- ☐ Narrow mobile screens do not clip chart labels, legends, or metric values.

6. Maintenance jobs

Create Job (/jobs/create-job)

- ☐ **Create Job** loads location options from the API.
- ☐ Selecting a location filters Area; selecting Area filters Equipment; selecting Equipment filters Asset ID.
- ☐ Changing a parent selection clears incompatible child selections.
- ☐ Submit with required fields missing; Location, Type, Priority, Equipment, Breakdown Time, Impact, and Description errors appear.
- ☐ Create a valid single-asset job and verify the generated request/job-card identifier, pending status, requester, timestamps, and data.
- ☐ Add two or more assets to one job; each asset preserves its own area, equipment, asset ID/reason, notes, and images.
- ☐ Remove a middle asset from a multi-asset request; remaining values stay attached to the correct assets.
- ☐ Attempt to remove the last asset; at least one asset remains or validation blocks submission.
- ☐ Select an equipment item with a valid barcode; the Asset ID list and stored ID are correct.
- ☐ Test the no-barcode flow with each reason: no barcode visible, damaged barcode, rental unit, and other.
- ☐ Choose **other** without details; submission is blocked. Add details and confirm it succeeds.
- ☐ Choose a no-barcode reason without an image; submission is blocked because evidence is required.
- ☐ Upload one and multiple supported images; previews, filenames, compression, metadata submission, presigned uploads, and later display work.
- ☐ Try an unsupported, oversized, corrupt, duplicate, and zero-byte image; the UI handles each safely.
- ☐ Use a past, current, and future Breakdown Time; confirm the accepted rule matches business expectations.
- ☐ Cancel creation; no record or orphaned upload remains.
- ☐ Simulate metadata success followed by S3 upload failure; the user receives an actionable error and the system does not silently report full success.

Pending approval (/jobs/pending-approval, admin)

- ☐ Pending list shows only pending requests with correct columns/status badges.
- ☐ Search, column filters, pagination, page size, sorting, and row actions work together.
- ☐ Open a request; all common and per-asset details/images display correctly.
- ☐ Switch among assets in a multi-asset request; approval details track the selected asset without losing shared request information.
- ☐ Approve without assignee/group/target date; validation prevents submission.

- ☐ Approve and assign to a technician/contractor as supported; assignee name, group, approver, approval timestamp, target date, and status persist.
- ☐ Attempt an invalid/past target date; confirm the business rule is enforced.
- ☐ Reject without a reason; validation prevents submission.
- ☐ Reject with a reason; status, reject message, rejected-by, and rejected-at persist and display.
- ☐ Cancel/close approve and reject dialogs; no mutation occurs.
- ☐ Confirm approve/reject actions disappear or become invalid after the first decision, including from a second browser tab.
- ☐ Delete a disposable pending request; Cancel preserves it and Confirm removes it after list refresh.

Open/in-progress jobs (/jobs/in-progress)

- ☐ Approved work appears in the in-progress list for the correct permitted users/assignee.
- ☐ Open job details; request data, assignment, target date, status, and images are correct.
- ☐ Overdue highlighting is correct at the date boundary and uses the intended timezone.
- ☐ Edit/update an allowed request; changed fields persist after refresh without losing existing images.
- ☐ Delete/cancel an in-progress job only if the business rules permit it; unauthorized roles cannot do so.
- ☐ Open **Comments** from a row; the sidebar is tied to the correct request.
- ☐ Open **Action Job** and verify required fields: start/end time, total km, work completed, root cause, status, signature, and signed-by.
- ☐ Enter future start/end times and an end before start; validation blocks each invalid case.
- ☐ Enter zero, negative, non-numeric, and valid distance/cost values; validation and stored values are correct.
- ☐ Test optional findings, sundries, parts, contractor, costs, work-order number, images, and invoices.
- ☐ Draw a signature, clear it, redraw it, and submit; missing signature is blocked and the saved signature is correct.
- ☐ Submit a completed action; the job leaves Open Jobs and appears once in Completed Jobs.

Completed jobs (/jobs/completed and /jobs/:id/complete)

- ☐ List scope is correct: admin sees intended completed jobs; other roles see only their intended jobs.
- ☐ Completed details combine request and action data correctly, including costs, root cause, evidence, assignee, requester, and timestamps.
- ☐ Image/invoice links open the correct objects and expired/missing links fail gracefully.
- ☐ Download the job card PDF; filename, job number, fields, images/signature, formatting, and totals are correct.
- ☐ Repeated download does not mutate the job or open duplicate uncontrolled windows.
- ☐ Job status transitions cannot be replayed to produce duplicate action records.

Comments

- ☐ Open comments from each job list/detail action; comments belong to the selected request only.
- ☐ Post an empty/whitespace comment; it is rejected.
- ☐ Post a normal, long, multiline, punctuation, emoji, and HTML-like comment; content is stored/displayed safely without script execution.
- ☐ A new comment appears immediately with correct author and timestamp and remains after reload.
- ☐ Simultaneous comments from two users appear without one overwriting the other.

- ☐ If WhatsApp comment notifications are enabled in the test environment, confirm intended recipients receive exactly one message with job reference, comment, author, and timestamp.
- ☐ A WhatsApp delivery failure does not make the successfully stored comment appear failed; failure is observable server-side.

7. Assets

Create New Asset (</assets/create-new-asset>, admin)

- ☐ The form loads Location, Business Unit, Area, Equipment, Asset Type, Category, Condition, and dependent option data.
- ☐ Submit with required fields missing; clear errors appear.
- ☐ Create a valid General asset with Location, Business Unit, Area, Equipment, Asset ID, Condition, Category, Replacement Value, optional Serial Number, Notes, and Images.
- ☐ Create a Low Value asset below R5,000; it succeeds without an Asset ID if that is the intended rule.
- ☐ Set a Low Value replacement value at/above R5,000; validation blocks it at the exact boundary expected by the business.
- ☐ Create a High CAPEX asset above R50,000; it succeeds.
- ☐ Set a High CAPEX replacement value at/below R50,000; validation blocks it at the expected boundary.
- ☐ Create a Rental asset without an Asset ID; it follows the intended rule.
- ☐ For asset types requiring a barcode, omit Asset ID; validation blocks submission.
- ☐ Test zero, negative, decimal, very large, and formatted replacement values; stored currency/value is accurate.
- ☐ Attempt a duplicate Asset ID and duplicate Serial Number; the system applies the intended uniqueness rules with a useful error.
- ☐ Upload multiple supported images and test invalid/oversized/corrupt/duplicate files.
- ☐ Cancel creation; no record/orphaned file remains.

Asset register (</assets/list>, admin)

- ☐ Newly created assets appear with correct values after creation and hard refresh.
- ☐ Desktop table and mobile cards show equivalent data and actions.
- ☐ Search and filter by Location, Equipment, verification status, and other exposed columns.
- ☐ Combine filters, clear/reset them, paginate, change page size, and sort; results/counts remain correct.
- ☐ Column visibility selection works and does not break row actions.
- ☐ Open **View**; the correct asset detail loads.
- ☐ Open **Edit**; current values and existing images are prefilled.
- ☐ Update text/select/value fields; save and confirm list/detail refresh.
- ☐ Keep all existing images while updating metadata; no image is lost.
- ☐ Add new images, remove selected existing images, and combine add/remove in one update; S3 and UI results match.
- ☐ Cancel update; no changes persist.
- ☐ Delete an asset using Cancel and Confirm paths; confirm list removal and intended file/history behavior.
- ☐ Attempt to delete an asset referenced by jobs/transfers/disposals; the business rule prevents orphaned records or clearly explains the result.

Asset details and history (</assets/:id>, </assets/:id/history>)

- ☐ Detail tabs show Details, Verification, Jobs, and Transfers data for the same asset.
- ☐ Image gallery opens full-screen, moves between images, closes with button/Escape, and handles no images.
- ☐ History metrics show completed, in-progress, pending, total cost, MTBF, MTTR, availability, and failure count accurately.
- ☐ Maintenance cost chart and job history match known completed-job data.
- ☐ A history row opens the correct completed job details.
- ☐ An asset with no history shows a stable empty state and zero/null metrics appropriately.

Barcode and manual verification

- ☐ Admin and Manager can enter </assets/verification>; other roles cannot.
- ☐ Grant camera permission; scanner starts, shows a usable camera view, and scans a known test barcode once.
- ☐ Deny camera permission; the page explains recovery instead of hanging/crashing.
- ☐ Test no camera, camera already in use, front/rear camera, slow initialization, and leaving the page while scanning.
- ☐ Scan a valid Asset ID; verification posts the captured latitude/longitude and shows success.
- ☐ Scan an unknown, malformed, or repeated barcode; controlled error/de-duplication behavior occurs.
- ☐ Grant and deny geolocation permission; both outcomes are handled clearly.
- ☐ Confirm coordinates are not submitted as zero/stale values before geolocation resolves.
- ☐ Verify status, verified-by, last-verified date, next-due date, and location update in list/detail/history.
- ☐ Open **Manual Verification** from an asset; selected asset is correct.
- ☐ Submit manual verification without Location, Reason, or an Image; each required-field error appears.
- ☐ Submit valid manual verification; evidence uploads and verification state updates.
- ☐ Ensure manual verification cannot accidentally verify a different asset after switching rows/dialogs.

8. Asset transfers

Create Transfer (</transfers/create-new-transfer>)

- ☐ Location From and Location To options load.
- ☐ Select the same From and To location; validation blocks submission.
- ☐ Select From, then Area, Equipment, and Asset ID; dependent options show only eligible source assets.
- ☐ Changing Location From clears incompatible selected assets and child fields.
- ☐ Create a valid one-asset transfer with reason and expected date.
- ☐ Create a multi-asset transfer; add/remove/reorder behavior keeps each asset's data/images correct.
- ☐ Test no-barcode reasons, required "other" details, and required image evidence.
- ☐ Try an expected date in the past and at the boundary; the intended business rule is enforced.
- ☐ Prevent the same asset being added twice to a single request.
- ☐ Cancel creation; no request/upload remains.

Transfer requests (</transfers/requests>)

- ☐ List and mobile cards correctly distinguish pending, approved, rejected, and cancelled requests.

- ☐ Open request details and move among multiple assets; source/destination, requester, dates, reason, evidence, and status are correct.
- ☐ Approve a pending transfer; decision metadata and status persist and the item becomes eligible for transit.
- ☐ Reject without a reason; validation blocks it. Reject with a reason; rejection details persist.
- ☐ Cancel a request where allowed; a reason is required and the item no longer progresses.
- ☐ Only valid actions appear for each status; stale tabs cannot repeat a decision.
- ☐ Delete only when permitted; Confirm/Cancel and refresh behavior are correct.

Mark in transit (/transfers/:id/in-transit)

- ☐ Only an approved transfer can enter transit.
- ☐ Submit without Transport Type, Transport By, or Transport Date; validation appears.
- ☐ Selecting courier exposes Tracking Number and enforces its intended requirement.
- ☐ Test contractor, employee, other, and courier transport types.
- ☐ Enter zero, negative, decimal, and valid Transport Cost.
- ☐ Test future/past Transport Date according to the intended rule.
- ☐ Upload transport images and invoices; files remain linked to the correct transfer.
- ☐ Successful submission changes status to **in-transit**, removes it from actionable requests, and adds it to In Transit.
- ☐ Re-submitting from a stale page does not create duplicate transit data.

Receive transfer (/transfers/:id/receipt)

- ☐ Only an in-transit transfer can be received.
- ☐ Enter Date Received and each condition: excellent, good, fair, damaged.
- ☐ Selecting damaged reveals Damage Details and blocks submission when details are missing.
- ☐ Add Notes, Images, and Delivery Note; verify successful uploads and later display.
- ☐ Receipt date before transit date or in the future is rejected as intended.
- ☐ Successful receipt sets status to completed and moves every transferred asset to Location To exactly once.
- ☐ Asset register/detail reflects the new location after refresh.
- ☐ Partial backend/upload failure does not leave transfer status and asset locations inconsistent without a visible recovery path.

In Transit and Completed lists

- ☐ /transfers/in-transit contains only in-transit items and correct receipt actions.
- ☐ /transfers/completed contains only completed transfers with full request/transit/receipt details.
- ☐ Admin versus non-admin "Completed Transfers/My Transfers" scoping is correct.
- ☐ Search, filtering, pagination, mobile cards, View, Delete, and other visible actions behave consistently.

9. Asset disposals

Create Disposal (/disposals/create-new-disposal)

- ☐ Location, Area, Equipment, Asset ID, Disposal Reason, Expected Disposal Date, and Description load/validate.

- ☐ Create a valid single-asset disposal request.
- ☐ Create a multi-asset disposal request; per-asset selection, no-barcode details, and images persist correctly.
- ☐ Prevent duplicate assets within a disposal and assets already in an incompatible active workflow.
- ☐ Test all available disposal reasons and any reason-specific fields.
- ☐ Test no-barcode reason, required "other" details, and required evidence images.
- ☐ Past/boundary Expected Disposal Date follows the intended business rule.
- ☐ Cancel creation; no request/orphaned upload remains.

Disposal requests (/disposals/requests)

- ☐ Pending, approved, rejected, and cancelled states display correctly on desktop and mobile.
- ☐ Request details show all assets, reason, description, requester, location, evidence, and dates.
- ☐ Approve a pending disposal and verify approver/decision metadata and status.
- ☐ Reject with and without a reason; validation and persisted rejection data are correct.
- ☐ Cancel where supported; cancellation reason/status persist.
- ☐ Only approved requests offer the Dispose/Complete action.
- ☐ Status transitions cannot be replayed from stale tabs.

Complete disposal (/disposals/:id/completed)

- ☐ Only an approved disposal can be completed.
- ☐ Disposal Method is required; test every available method.
- ☐ Test completion notes, costs/proceeds if displayed, evidence images, certificates/invoices, date, and other exposed fields.
- ☐ Successful completion sets status to **disposed** and moves the record to Completed Disposals.
- ☐ Disposed assets are no longer offered for new jobs/transfers/disposals where business rules exclude them.
- ☐ Asset register/detail represents disposed state or removal according to the intended policy.
- ☐ Partial file/API failures do not silently leave contradictory asset/disposal states.

Completed disposals

- ☐ /disposals/completed contains only disposed records and intended user scope.
- ☐ Completed detail tabs/sections show Request, Disposed, and Costs data correctly.
- ☐ Download the disposal document PDF; filename, assets, decision, completion data, costs, evidence, and formatting are correct.
- ☐ Search, filtering, pagination, View, Download, Delete, and mobile equivalents work as intended.

10. Users and profiles

User list (/users, admin)

- ☐ List loads created date, name, location, position, email, mobile, group, account status, and updated timestamp.
- ☐ Search, sorting, pagination, page-size changes, empty state, and mobile cards work.
- ☐ **Resend Password** appears only for **FORCE_CHANGE_PASSWORD** users.
- ☐ Resend temporary password delivers credentials once, provides safe feedback, and never displays/logs the password.

- ☐ Confirmed users do not expose an invalid resend action.

Create User (/users/create-user, admin)

- ☐ Submit empty fields; Name, Surname, Location, Group, Position, Email, and Mobile errors appear.
- ☐ Test invalid and valid email formats.
- ☐ Test invalid South African mobile values and valid 0[6-8]XXXXXXXX values.
- ☐ Create one user for every group: admin, manager, user, maintenance, and contractor.
- ☐ Verify Cognito identity, group membership, application profile, timestamps, and initial FORCE_CHANGE_PASSWORD status are all created.
- ☐ Verify temporary credentials are delivered through the configured channel and not returned in API/UI/logs.
- ☐ Attempt duplicate email/username; no duplicate or half-created profile remains.
- ☐ Simulate Cognito success plus database failure, and database success plus Cognito/group failure; retry/reconciliation is safe and visible.
- ☐ Cancel creation; no user is created.

Admin user profile (/users/:id)

- ☐ View a selected user and confirm the route displays that user, not the signed-in admin.
- ☐ Update each editable attribute and confirm Cognito/application profile synchronization where required.
- ☐ Changing group removes old privileges and grants new ones after session/token refresh.
- ☐ Invalid email/mobile and duplicate identity values are rejected cleanly.
- ☐ Cancel update; values remain unchanged.
- ☐ Delete a disposable user: Cancel preserves it; Confirm removes/disables both Cognito identity and application profile as intended.
- ☐ Deleting the current admin or last admin is prevented if required by business policy.
- ☐ A deleted user cannot log in and is removed from lists after refresh.

My Profile (/users/profile)

- ☐ Each role sees only its own name, surname, location, group, position, email, mobile, and metadata.
- ☐ Editing Mobile Number accepts a valid number and persists after reload.
- ☐ Invalid mobile numbers are rejected.
- ☐ A user cannot alter disabled identity/authorization fields via the UI or crafted request.
- ☐ A user cannot change another profile by modifying the route/request ID.

11. Notifications

- ☐ Notification button opens/closes the sidebar and shows a correct unread count.
- ☐ All, Unread, Read, and Archived tabs/categories show the correct notifications and counts.
- ☐ Trigger transfer submitted/approved/rejected/in-transit/received/cancelled notifications and verify recipients, title, message, priority, time, and link target.
- ☐ Trigger job assigned/completed/overdue notifications and verify the same fields.
- ☐ Mark an unread notification read; badge and all counts update immediately and after reload.
- ☐ Archive a notification if supported; it leaves active lists and appears in Archived.

- ☐ Opening a notification routes to the correct existing record and handles deleted/inaccessible records safely.
- ☐ Two notifications arriving close together are both retained and ordered newest-first.
- ☐ Duplicate backend delivery does not display unintended duplicates.
- ☐ Timestamps use the intended local timezone and readable relative/absolute format.
- ☐ Notification failure never blocks the underlying job/transfer/disposal transaction.
- ☐ Users cannot fetch or mutate another recipient's notifications.

12. Tables, search, filters, and data consistency

Run these checks on Assets, Jobs, Transfers, Disposals, Users, and any supported Stock list.

- ☐ Search is case-insensitive as intended and handles spaces, punctuation, partial values, and no results.
- ☐ Clear search restores the full result set.
- ☐ Each column sort works ascending/descending and handles empty values.
- ☐ Multiple filters combine correctly and Reset restores defaults.
- ☐ Pagination never shows an empty page after deleting the last row on a page.
- ☐ Page size responds to viewport/selector without duplicate or missing rows.
- ☐ Row actions always target the clicked row after sorting/filtering/pagination.
- ☐ Rapidly opening actions on different rows never reuses the previous row ID.
- ☐ Desktop table and mobile cards expose equivalent valid actions for each status/role.
- ☐ A mutation in one tab/list invalidates or refreshes all related cached views.
- ☐ Dates, currency, names, statuses, and identifiers use consistent formatting across list, detail, PDF, and notification.
- ☐ Null, missing, unusually long, and Unicode backend values do not crash or break layout.

13. Responsive and cross-browser testing

Test at approximately 320 px, 375 px, 768 px, 1024 px, and a wide desktop viewport.

- ☐ Complete each critical workflow on desktop and mobile: login, create job, approve job, action job, create/approve/receive transfer, create/approve/complete disposal, asset verification, and profile update.
- ☐ No horizontal page scrolling, clipped buttons, overlapping fields, hidden dialog actions, or unreadable tables/cards.
- ☐ Forms remain usable when the mobile keyboard is open.
- ☐ Dialogs, image viewer, chat, notifications, and sidebar fit the viewport and scroll internally where needed.
- ☐ Orientation changes do not lose unsaved form values or trap overlays.
- ☐ Test current Chrome, Edge, Firefox, and Safari/iOS where available.
- ☐ Browser Back/Forward preserves sensible route/state behavior without replaying mutations.
- ☐ Zoom to 200%; content and controls remain usable.

14. Accessibility and usability

- ☐ Complete login and one full critical workflow using keyboard only.
- ☐ Tab order follows the visual order; no keyboard trap occurs in dialogs or sidebars.
- ☐ Escape closes modal/sidebar surfaces where expected without discarding data silently.

- ☐ Inputs have visible labels; icon-only buttons have accessible names/tooltips.
- ☐ Validation messages are associated with fields and are announced to assistive technology.
- ☐ Focus moves to the first invalid field or a useful error summary after failed submission.
- ☐ Status is not communicated by color alone.
- ☐ Light and dark themes have readable contrast for text, badges, disabled controls, charts, and errors.
- ☐ Images have suitable alternative text; decorative images are ignored by screen readers.
- ☐ Loading and success/error state changes are perceivable without relying only on animation.

15. Reliability, security, and adverse conditions

- ☐ Throttle the network to Slow 3G; loading states work and no request is submitted twice.
- ☐ Go offline before loading a list and during a form submission/upload; the UI reports the problem and supports a safe retry.
- ☐ Return 400, 401, 403, 404, 409, 413, 429, and 500 responses from test endpoints; each produces correct, non-sensitive feedback.
- ☐ A 401 signs the user out once without a redirect loop.
- ☐ A 409 duplicate/conflict preserves form data for correction.
- ☐ A 413 file-too-large response identifies the file/problem.
- ☐ A 429 response discourages repeated submission and recovers after the limit window.
- ☐ Slow or failed presigned URL upload does not leave the UI permanently loading.
- ☐ Expired presigned download/image URLs are refreshed or fail with a controlled placeholder.
- ☐ Enter HTML/script/SQL-like strings into text fields, comments, search, filenames, and rejection reasons; values display as text and do not execute.
- ☐ Attempt IDOR by changing IDs in asset, job, transfer, disposal, user, comment, PDF, and notification requests; backend authorization blocks access.
- ☐ Confirm secrets, tokens, personal data, signatures, and presigned URLs are not unnecessarily logged.
- ☐ Confirm HTTPS-only behavior and secure cookie/token practices in the production-equivalent environment.
- ☐ Submit the same mutation concurrently in two tabs; status transition and data remain idempotent/consistent.
- ☐ Set the client clock/timezone incorrectly; server-owned audit timestamps and authorization remain trustworthy.

16. Data integrity and end-to-end scenarios

Maintenance lifecycle

- ☐ Create an asset → create a multi-asset job → approve/assign it → comment → action/complete it → download PDF → verify asset history and dashboard metrics update.

Transfer lifecycle

- ☐ Create an asset at Location A → request transfer to Location B → approve → mark in transit → receive → verify completed record, notification trail, and asset Location B.

Disposal lifecycle

- ☐ Create an asset → request disposal → approve → complete disposal → download document → verify the asset cannot incorrectly enter a later active workflow.

User lifecycle

- ☐ Admin creates user → invitation arrives → first-login password change → correct role navigation → My Profile update → admin update/resend rules → deletion/disable → login denied.

Cross-module consistency

- ☐ The same Asset ID, Location, Equipment, user name, job number, transfer ID, and disposal ID remain consistent across lists, details, history, comments, notifications, and PDFs.
- ☐ Deleting or changing an upstream record does not leave broken links or misleading historical audit data.
- ☐ Dashboard metrics reconcile after all lifecycle scenarios and a hard refresh.

17. Known route/scope checks to resolve before release

These are not assumed defects, but the current source makes them important acceptance decisions.

- ☐ **Login redirect:** Verify an API 401 redirects to the real public login route `/`; record/fix any redirect to the non-routed `/login`.
- ☐ **Task History:** The sidebar exposes `/jobs/actioned` to some roles, but no matching route is defined; decide whether to implement, hide, or explicitly exclude it.
- ☐ **Transfer Requests visibility:** The sidebar advertises Transfer Requests to several roles while the route is admin-guarded; align intended visibility and authorization.
- ☐ **Disposal Requests visibility:** The sidebar advertises Disposal Requests to several roles while the route is admin-guarded; align intended visibility and authorization.
- ☐ **Contractor profile:** Confirm contractor group recognition and `/users/profile` access agree with the intended policy.
- ☐ **Manual verification URL:** Confirm `/assets/verification/manual` intentionally renders the asset register and opens the expected workflow.
- ☐ **User update endpoint:** Confirm admin and self-service updates call the intended `/api/users...` routes consistently.
- ☐ **Environment variable:** Confirm deployment and localhost use the same intended API variable (`VITE_SITE_URL` versus any `VITE_PUBLIC_API_URL` deployment setting).
- ☐ **Unknown route:** Confirm there is an intentional Not Found/access-denied experience for invalid URLs.

18. Production migration gate

Do not migrate until every required item below is satisfied.

- ☐ All P0/P1 critical workflows pass for every applicable role on desktop and mobile.
- ☐ No unresolved authorization, data-loss, duplicate-transaction, broken-upload, or cross-module consistency defect remains.
- ☐ Lint and production build pass.
- ☐ Test data and orphaned S3 objects/Cognito users are cleaned from the test environment.

- ☐ Production environment variables, API URL, Cognito IDs, S3/CORS settings, and allowed origins are independently reviewed.
- ☐ Backup/rollback procedure is documented and rehearsed.
- ☐ Monitoring exists for API/Lambda failures, Cognito failures, upload failures, notification failures, and frontend errors.
- ☐ A post-deployment smoke test owner and rollback decision-maker are assigned.
- ☐ Final business owner sign-off is recorded below.

Approval	Name	Date	Result / Notes
QA			
Product/Operations			
Technical			

Test execution log

ID	Section/test	Role	Browser/device	Result	Defect/evidence	Notes
1						
2						
3						